

文献阅读笔记

Efficient Memory Management for Large Language Model Serving with PagedAttention

vLLM 与 PagedAttention 经典论文精读

一句话抓住论文

vLLM 把操作系统的“虚拟内存 + 分页 + Copy-on-Write”迁移到 LLM 的 KV cache 管理，用非连续、按需分配的固定大小块显著减少显存碎片与重复，从而扩大 batch 并提升服务吞吐。

作者	Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica
会议	SOSP 2023 (29th Symposium on Operating Systems Principles)
标识	DOI: 10.1145/3600006.3613165 arXiv: 2309.06180
阅读版本	arXiv v1, 2023-09-12, 共16 页
笔记生成日期	2026-09-26

阅读标记：【论文原文】表示作者明示结论；【批判性推断】表示基于设计和实验边界的分析。

01 一页式摘要

先建立整体认知，再进入机制与实验。

研究问题

在自回归 LLM 在线服务中，KV cache 随序列逐 token 增长，长度又不可预知。连续、预分配的显存管理会产生预留浪费、内部碎片和外部碎片，使显存先于计算成为 batch size 的瓶颈。

维度	核心答案	原文证据
方法	PagedAttention: 将每个序列的 KV cache 分成固定 token 数的 blocks, 通过 block table 把逻辑块映射到非连续物理块。	Section 4.1-4.3, Fig. 5-7
系统	vLLM: 中央调度器 + KV cache manager + GPU workers; 实现按需分配、引用计数、Copy-on-Write、抢占恢复。	Section 4, Fig. 4
结果	vLLM 在同等延迟水平下较 FasterTransformer/Orca 提高 2-4× 吞吐; 长序列、大模型、复杂解码中收益更大。	Abstract; Section 6
代价	PagedAttention 的块表间接寻址使 attention kernel 延迟比高度优化的 FasterTransformer 高 20%-26%。	Section 7.1, Fig. 18a

因果链

动态 KV cache	→	碎片/预留浪费	→	可容纳请求数下降	→	吞吐下降
PagedAttention	→	非连续+按需分配	→	batch 变大	→	吞吐提升

来源: Abstract; Section 1, pp. 1-2; Section 7.1, p. 12.

02 问题为什么难

真正的瓶颈不是“没有显存”，而是“显存无法被高效拼接和共享”。

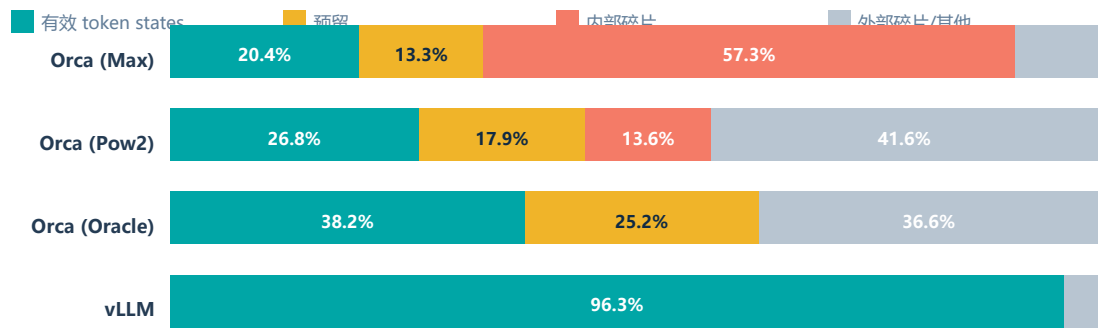
原文给出的关键例子：对 OPT-13B，单 token 的 KV cache 为：

$2 (K,V) \times 5120 \text{ (hidden size)} \times 40 \text{ (layers)} \times 2 \text{ bytes (FP16)} = 800 \text{ KB/token}$
当最大序列长度为 2048 时，单请求 KV cache 可达约 1.6 GB。A100 40GB 上，13B 模型参数约占 26GB，KV cache 仅能分到约 12GB。

来源：Section 3, p. 4; Table 1, p. 9.

连续预分配的三种浪费

类型	含义	为什么伤害吞吐
预留 (reservation)	为尚未生成的 token 预留空间。	请求存活期间其他请求无法使用，虽未必永久浪费，但限制同时 batch。
内部碎片	请求实际结束长度小于预分配大小。	最终确定不会存储 token，属于纯浪费。
外部碎片	不同连续块之间的不可利用空隙。	总空闲量可能足够，但没有足够大的连续区域。



读图：vLLM 将真正存储 token 状态的占比提高到 96.3%。

来源：Figure 2 and Section 3.1, pp. 2, 4. Orca (Oracle) 数值中未单独显示内部碎片，其余归入外部碎片/其他。

03 PagedAttention 的机制

本质是在 attention 算子中引入从逻辑位置到物理 block 的间接寻址。

逻辑 KV blocks (请求视角)



物理 KV blocks (GPU DRAM)



关键：逻辑连续 ≠ 物理连续；按需分配，最多只浪费一个未填满块的尾部空间。

操作系统概念	vLLM 对应物	直接效果
page	KV block (固定数量 token 的 K/V 向量)	统一大小，消除外部碎片
page table	block table	逻辑连续可映射到非连续 GPU 内存
demand paging	按需分配新物理 KV block	不再为最大长度预留整块空间
fork + refcount	多样本/多 beam 共享 block	避免重复 KV cache
copy-on-write	写入共享块时才复制一个 block	保持语义正确且将复制最小化

分块 attention (原文 Eq. 4 的含义)

对第 i 个 query, kernel 按 block 读取 K_j 与 V_j , 计算该块的 attention score A_{ij} , 再对所有已用 blocks 汇总得到输出 o_i 。算法不要求 K/V 在物理内存中连续，但需要查 block table。

$$A_{ij} = \text{softmax}(q_i^T K_j / \sqrt{d}); o_i = \sum_j V_j A_{ij}^T$$

块大小 B 是系统折中：小 block 减少尾部碎片、增加共享机会，却带来更多元数据和小颗粒访存；大 block 反之。作者默认 $B = 16$ tokens。

来源：Section 4.1-4.3, pp. 5-6; Section 7.2, p. 12.

04 vLLM 系统如何跑起来

论文的贡献不只是一个 kernel，而是调度、内存与执行的协同设计。



每一个 decoding iteration 重复；调度与内存管理是协同设计。

单序列解码

- Prefill 时只为 prompt 所需的逻辑块分配物理块，不预留最大生成长度。
- 每次 decoding 生成新 token，若最后一块有空位则直接追加；若已满则申请一个新物理块并更新 block table。
- 请求结束后立即释放其 blocks。因为只有最后一块可能未填满，单请求的浪费上界是一个 block。

来源：Section 4.3, Fig. 6-7, p. 6.

复杂解码与共享

场景	共享策略	关键操作
Parallel sampling	多个输出共享 prompt blocks；分叉后的生成状态独立。	refcount + block-level Copy-on-Write
Beam search	候选之间共享不只是 prompt，共享模式随 beam 裁剪动态变化。	fork / free / refcount，只在写入旧共享块时复制
Shared prefix	服务商预先缓存公共 system prompt 的 KV blocks。	请求把逻辑块指向缓存块，只计算任务输入

调度与抢占

- 采用 FCFS 保证公平；需抢占时，后到请求先被抢占。
- 驱逐采用 all-or-nothing：同一 sequence 的所有 blocks 一起驱逐；同一请求内多个 sequence 成组调度。
- 恢复方式有 CPU swapping 和 recomputation。中等 block size 16-64 时两者端到端性能接近。

来源：Section 4.4-4.6, pp. 7-9; Section 7.3, p. 13.

05 实现与工程细节

为了抵消分页寻址的开销，vLLM 需要定制 GPU kernels，而不是只替换分配器。

模块	实现	意义
前端	FastAPI，扩展 OpenAI API 接口和采样参数。	将系统做成完整在线服务。
控制平面	Python：调度器、block manager。	易于表达动态请求与块表状态。
数据平面	C++/CUDA：PagedAttention 及块写/读/拷贝内核。	降低 kernel launch 和非连续访存开销。
规模	8.5K 行 Python + 2K 行 C++/CUDA。	原型代码规模较紧凑。

三个关键 kernel 优化

1 Fused reshape + block write	把 KV 拆块、重排为读取友好布局并写入 block table 指定位置，融合成单个 kernel。
2 Fused block read + attention	修改 FasterTransformer attention kernel，边查 block table 边读 KV 并计算 attention；每个 warp 读一块以保证 coalesced access。
3 Fused block copy	将多个不连续 block 的 Copy-on-Write 拷贝合并到一次 kernel launch，避免大量 cudaMemcpyAsync 小传输。

分布式执行

vLLM 支持 Megatron-LM 式 tensor parallelism。每个 worker 只保存对应 attention heads 的 KV cache，但所有 workers 共享同一份由中央调度器维护的逻辑到物理 block 映射。每步调度器广播 token IDs 和 block tables，workers 在模型内部用 all-reduce 同步。

来源：Section 4.6 and Section 5, p. 9.

工程上的真正创新

“分页”不是一个独立数据结构。只有当 attention kernel、block manager、调度器、复杂解码语义和分布式执行共同遵守块映射时，内存效率才能转化为端到端吞吐。

06 实验设计与评价指标

解读结果前，先看它比了什么、在什么硬件上比、怎么判定达到容量上限。

模型	服务器	GPU 总显存	参数大小	KV cache 显存	KV slots
OPT-13B	1× A100	40 GB	26 GB	12 GB	15.7K
OPT-66B	4× A100	160 GB	132 GB	21 GB	9.7K
OPT-175B	8× A100-80GB	640 GB	346 GB	264 GB	60.1K

工作负载

数据集	平均输入	平均输出	特点
ShareGPT	161.31 tokens	337.99 tokens	长、方差大；用户分享的 ChatGPT 对话
Alpaca	19.31 tokens	58.45 tokens	短指令数据；由 GPT-3.5 self-instruct 生成

请求到达时间由 Poisson 分布合成；多数实验使用 1 小时 trace，OPT-175B 因成本限制使用 15 分钟 trace。

基线

基线	设定	解读时的意义
FasterTransformer	高度优化的分布式推理引擎；作者补了 dynamic batching scheduler。	代表 kernel 快但调度/内存管理粗粒度的系统。
Orca (Max)	一律为最大序列长度 2048 预留输出空间。	现实但悲观的连续预留。
Orca (Pow2)	预留到真实输出长度的下一个 2 的幂。	最多 2× 过度预留。
Orca (Oracle)	事先知道真实输出长度。	不可实现的 Orca 性能上界；对 vLLM 是强基线。

核心指标：Normalized latency = 所有请求的 (end-to-end latency / output length) 的均值。查看在保持低 normalized latency 时能承受多高 request rate；超过服务容量后，队列与延迟会发散。

来源：Section 6.1, Table 1 and Fig. 11, pp. 9-10.

07 主要结果：吞吐提升从哪里来

证据显示，内存效率通过“同时容纳更多请求”转化为系统吞吐。

场景	定量结果	解释
Basic sampling / ShareGPT	vLLM 比 Orca (Oracle) 可承受 1.7-2.7× 更高请求率；比 Orca (Max) 高 2.7-8×；比 FasterTransformer 最高 22×。	长序列下预留与碎片影响大；精细调度与分页内存叠加收益。
OPT-13B batch 数	ShareGPT: vLLM 30.42, Orca Oracle 13.62, Orca Max 7.00。Alpaca: vLLM 132.44, Oracle 72.75, Max 7.00。	在同一时刻处理的请求更多，直接验证了吞吐提升的中间机制。
Parallel sampling	Alpaca 上通过 prompt block 共享节省 6.1%-9.8% 内存；ShareGPT 上节省 16.2%-30.5%。	输入越长，共享 prompt 的价值越高。
Beam search	Alpaca 上节省 37.6%-55.2%；ShareGPT 上节省 44.3%-66.3%。beam width=6 时，比 Orca (Oracle) 的提升由 basic sampling 的 1.3× 增至 2.3×。	中间前缀可持续共享，不只是 prompt，所以收益更高。
Shared prefix	1-shot (80 tokens) 时比 Orca Oracle 高 1.67×；5-shot (341 tokens) 时高 3.58×。	前缀越长，缓存和共享能避免的重复计算/存储越多。
Chatbot	vLLM 可承受约 2× 于三种 Orca 的请求率。	长对话导致 prompt 多被截到 1024 tokens，Orca 还需为输出预留 1024 slots。

不要把“2-4×”理解为单次前向更快

PagedAttention 本身的 attention kernel 反而慢 20%-26%。端到端吞吐提升来自更高的显存利用率、更大的 batch 与更高的 GPU 利用率。这是一个“牺牲局部算子最优，换取全局系统最优”的典型例子。

来源：Section 6.2-6.5, Fig. 12-17, pp. 10-12; Section 7.1, p. 12.

08 消融实验与设计折中

消融部分说明了分页机制的代价、block size 的选择与抢占恢复方案。

问题	实验结论	工程决策
间接寻址多贵？	PagedAttention attention kernel 比 FasterTransformer 高 20%-26% 延迟。	通过融合读取与 attention，且把代价限制在 attention 而非 Linear 等全模型算子中。
block size 多大？	ShareGPT 上 16-128 表现好；Alpaca 序列短，大 block 显著变差。	默认 16 tokens：足够利用 GPU 并行，又避免显著尾部碎片。
抢占后如何恢复？	small block 下 swapping 因小颗粒 PCIe 传输开销大；大 block 下 swapping 更占优；16-64 时端到端性能接近。	根据 GPU 算力、CPU-GPU 带宽和 block size 选 recomputation 或 swapping。

什么时候 vLLM 优势会缩小？

- 当可用 KV cache 显存很大、序列很短时，系统从 memory-bound 转为 compute-bound。原文 OPT-175B + Alpaca 就出现了 vLLM 相对 Orca Oracle/Pow2 优势较小的情况。
- 非 LLM DNN 服务若 tensor shape 静态或计算密集，分页的间接寻址开销可能大于内存收益。

来源：Section 6.2, p. 11; Section 7, pp. 12-13; Section 8, p. 13.

设计决策矩阵

负载特征	更合理的设计倾向
长 prompt / 长 output / 高方差	更小 block，优先减少碎片和预留。
多样本 / 大 beam width / 共享 system prompt	强化 refcount、Copy-on-Write 和 prefix cache，共享收益大。
频繁抢占 + 小 block	优先 recomputation，避免大量小块 PCIe 传输。
显存充足 + 短序列 + 计算密集	分页收益有限，应更关注 kernel/compute 优化。

09 创新、局限与批判性评价

把作者声称与我们基于证据的判断分开。

作者明示贡献

- 识别并定量 LLM serving 中 KV cache 分配的碎片、预留与不能共享问题。
- 提出能直接在非连续 KV cache 上计算的 PagedAttention。
- 实现分布式服务引擎 vLLM，将 block-level 内存管理、调度、共享和 GPU kernels 统一起来。
- 在多模型、多负载和多解码方式上证明较强的端到端吞吐收益。

来源: Section 1, pp. 1-2.

批判性限制

限制	为什么重要	证据标签
基线 Orca 为作者自行实现	Orca 未开源，自行实现可能与原系统存在工程差异；不过 Oracle 是很强的理论上界对照。	[论文原文] Section 6.1
请求时间是合成的	ShareGPT/Alpaca 只提供文本长度，到达过程用 Poisson 分布模拟，可能弱化了现实突发流量和多租户差异。	[论文原文 + 批判性推断] Section 6.1
终端指标不够全面	normalized latency 有利于比较吞吐，但没有完整呈现 TTFT、TPOT、p95/p99 和 SLO 违约率。	[批判性推断] 基于 Section 6.1 的指标定义
软硬件时代性	2023 年实验以 A100、OPT/LLaMA 为主；结论的机制仍重要，但绝对数据不能直接外推到新 GPU、GQA/MQA、超长上下文与量化。	[批判性推断] 基于 Table 1 与模型范围
块大小是静态选择	B=16 是在所测负载上的良好折中，未验证按负载/层/请求自适应块大小。	[批判性推断] 基于 Section 7.2

我的综合评价

这篇论文的持久价值不只是“vLLM 快”，而是它将一个新型工作集的语义特征（逐 token 增长、长度不可知、前缀可共享）映射到成熟的系统抽象（paging、refcount、Copy-on-Write），再为 GPU 执行特性重新工程化。

10 可复现清单与延伸实验

下列内容区分“原文已给出”和“今天重做时必须补充”。

原文可复现要点

类别	已给出信息	状态
代码	vLLM 源码公开；原文给出 GitHub 链接。	可用
模型	OPT-13B/66B/175B, LLaMA-13B。	明确
硬件	Google Cloud A2; A100 40GB 及 A100 80GB。	明确
软件	PyTorch, Transformers, NCCL, FastAPI; 定制 CUDA kernels。	组件明确，精确版本未在正文系统列出
负载	ShareGPT/Alpaca 长度分布, Poisson arrivals; 多数 1h, 175B 为 15min。	明确
核心参数	block size 默认 16; FCFS; 支持 recomputation/swapping。	明确

建议的现代化复现方案

- 锁定 vLLM 提交、CUDA/PyTorch 版本、GPU SKU 与驱动；同时报告 engine 参数与 memory utilization 上限。
- 同时测 TTFT、TPOT、E2E latency 的 p50/p95/p99、throughput 与 SLO attainment，避免仅用均值 normalized latency。
- 将长度与到达过程拆开控制：短/长 prompt，短/长 output，Poisson/突发/周期性流量。
- 测试不同 KV 结构：MHA、GQA、MQA；不同精度：FP16/BF16/FP8/INT8 KV cache。
- 对 block size 做 8/16/32/64/128 完整消融，同时报告有效 token state 占比、块表开销、kernel 延迟和端到端性能。

可以延伸的研究问题

方向	最小可行实验
自适应 block size	用在线长度分布与负载率动态选 B，对比固定 B=16 的 SLO 与碎片。
SLO-aware preemption	把 FCFS 换成具有 TTFT/TPOT 预算的调度，比较吞吐、尾延迟和公平性。
Prefix cache admission	在显存有限时决定哪些 prefix 值得常驻，评估命中率、重算与吞吐。
跨层/分层 KV 管理	针对不同层的访问与压缩特性采用不同放置/精度，测准确性-显存-延迟三方折中。

11 五分钟汇报稿与常见追问

可直接用于组会、课程汇报或答辩准备。

五分钟汇报稿

时间	要讲的内容
0:00-0:40	问题：LLM decoding 是 memory-bound，KV cache 随 token 增长且输出长度不可知。连续预分配使实际 token states 仅占 20.4%-38.2% KV 显存。
0:40-1:50	方法：PagedAttention 把 KV cache 分成固定大小 block，通过 block table 实现非连续物理存储与按需分配，对应 OS 虚拟内存。
1:50-2:50	系统：vLLM 统一调度器、KV manager 和 GPU workers；以 refcount + Copy-on-Write 支持 parallel sampling、beam search 和 shared prefix。
2:50-4:05	证据：在 OPT-13B/66B/175B 和 LLaMA-13B 上，vLLM 总体提升 2-4× 吞吐；ShareGPT 上比 Orca Oracle 承受 1.7-2.7× 更高 request rate。beam search 共享节省最高 66.3% 内存。
4:05-5:00	边界：attention kernel 本身慢 20%-26%，但全局更快；优势主要出现在 memory-bound、长序列和共享多的负载。局限包括合成到达过程、平均指标为主和 2023 年硬件/模型范围。

高频追问

问题	简答
PagedAttention 会改变模型输出吗？	不会。它改变 KV cache 的物理布局和读取方式，不改变 attention 的数学语义；原文称准确性不受影响。
为什么不直接做显存紧凑？	大型 KV cache 的搬移成本高；而且 compaction 不解决预分配和复杂解码中的共享问题。
为什么 block size 不越小越好？	过小 block 会增加块表与访存开销，降低 GPU 并行读取效率；默认 16 是实验中的折中。
FlashAttention 和 PagedAttention 是竞争关系吗？	不是简单替代。FlashAttention 主要优化 attention 计算的 I/O 与中间结果；PagedAttention 主要解决在线服务中持久 KV cache 的动态分配和共享。
这篇论文最重要的方法论是什么？	用工作负载语义重新设计系统抽象，并与 kernel 和调度协同优化；不要只追求单算子最快。

12 原文依据索引

用于回到论文快速核对数字、结论和图表。

本笔记中的主张	原文位置	证据类型	置信度
连续预分配只有 20.4%-38.2% KV 显存用于实际 token states。	Fig. 2; Section 1/3.1, pp. 2,4	图+正文	High
OPT-13B 单 token KV cache 约 800KB, 2048 tokens 约 1.6GB。	Section 3, p. 4	公式+文本	High
PagedAttention 允许连续逻辑 KV blocks 放在非连续物理内存。	Section 4.1, Eq. 4, Fig. 5, p. 5	公式+图+文本	High
单请求尾部浪费被限制在一个 block 内。	Section 4.3, p. 6	文本+机制	High
复杂解码用 refcount 和 Copy-on-Write 共享 KV blocks。	Section 4.4, Fig. 8-10, pp. 7-8	图+文本	High
vLLM 在同等延迟下总体提高 2-4× 吞吐。	Abstract; Section 1/6	摘要+实验	High
ShareGPT 上较 Orca Oracle 承受 1.7-2.7× 更高 request rate。	Section 6.2, Fig. 12, p. 11	实验图+文本	High
Beam search KV 共享最高节省 66.3% 内存。	Section 6.3, Fig. 15, p. 11	实验图+文本	High
Shared prefix 5-shot 下较 Orca Oracle 高 3.58×。	Section 6.4, Fig. 16, p. 12	实验图+文本	High
attention kernel 的间接寻址带来 20%-26% 额外延迟。	Section 7.1, Fig. 18a, p. 12	消融实验	High
默认 block size 为 16 tokens。	Section 7.2, Fig. 18b, p. 12	消融实验+文本	High
原评估未完整涵盖 TTFT/TPOT/p99/SLO。	Section 6.1 指标定义	批判性推断	Medium

参考文献

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23), 611-626. DOI: 10.1145/3600006.3613165. arXiv:2309.06180.

论文主页: <https://arxiv.org/abs/2309.06180>
DOI: <https://doi.org/10.1145/3600006.3613165>
项目: <https://github.com/vllm-project/vllm>

本笔记基于论文原文精读整理。定量结果均按原文报告；未在原文直接声称的判断已标为“批判性推断”。